

# Day 11 lab: inference on a network video stream

Naeemullah Khan

[naeemullah.khan@kaust.edu.sa](mailto:naeemullah.khan@kaust.edu.sa)



جامعة الملك عبد الله  
للعلوم والتقنية  
King Abdullah University of  
Science and Technology

KAUST Academy  
King Abdullah University of Science and Technology

October 3, 2026

- ▶ **Goal:** the Day 8 detector runs on a live network video stream, and you know the age of the image that it sees.
- ▶ **Boards:** the Raspberry Pi 5 with its camera, and the XIAOML Kit with its camera.
- ▶ **Deliverable:** a live demonstration, and the file `report.md` with the network values, the latency for two stream settings, the values of the XIAO stream, and the Decision Log.

## From the lecture to the lab

The lecture gave the network tools, the bit rates of MJPEG and H.264, RTSP, and the buffers that make a frame old. Its values came from one computer with no network and no camera. Today you measure the lab network, the two cameras, and the complete path from the camera to the boxes on the screen.

**Key point:** Measure the age of the image at the moment of the result, not only the time of the model.

# Plan of the three hours



| Part                    | Work                                                              | Time   |
|-------------------------|-------------------------------------------------------------------|--------|
| Quiz                    | The quiz of Week 2 (Days 6 to 10)                                 | 20 min |
| A                       | Network tools: find the devices, measure the throughput           | 20 min |
| B                       | RTSP server: publish the camera with MediaMTX, open the stream    | 40 min |
| C                       | Inference on the stream: the detector, the latency, two settings  | 45 min |
| D                       | Microcontroller camera: an MJPEG stream of the XIAO, a comparison | 25 min |
| Check by the instructor |                                                                   | 30 min |

Write each result in `report.md` when you get it.

## Hardware for each group

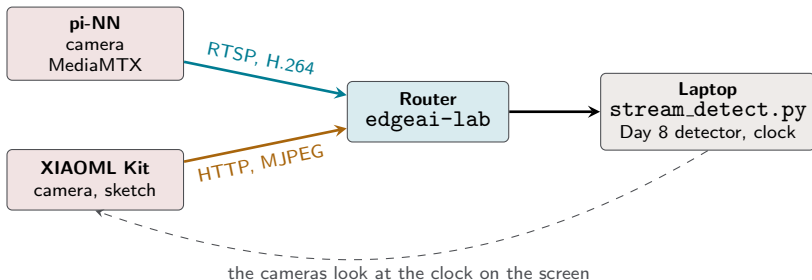
- ▶ Raspberry Pi 5 with the active cooler, the camera of Day 8, the card of your group, and the 27 W power supply.
- ▶ XIAOMI Kit with its camera, and a USB-C cable.
- ▶ Laptop in the lab network edgeai-lab.
- ▶ Optional: an Ethernet cable and a switch for the Raspberry Pi.

## Software

- ▶ Card: MediaMTX v1.21.1 in `~/mediamtx`, `iperf3`, `ffmpeg`.
- ▶ Laptop: the Day 8 environment `~/day08_env`, the Day 8 lab folder with `models/yolo11n.pt`, `iperf3`, `ffplay` or `VLC`.
- ▶ Arduino IDE with the esp32 core 3.3.12.

Lab files                      Labs/day11/ on the laptop. Run the laptop commands in this folder.

Names                        NN is your group: `pi-NN`, `X.(100+NN)`. The examples use `X = 192.168.8`.



|                       | Raspberry Pi                      | XIAO                        |
|-----------------------|-----------------------------------|-----------------------------|
| Address of the stream | rtsp://pi-NN.local:8554/cam       | http://<address>/           |
| Encoder               | H.264 in software (CPU)           | JPEG in the camera module   |
| Start setting         | 1280 × 720, 30 frames/s, 2 Mbit/s | 640 × 480 (VGA), quality 12 |

The model runs on the laptop: the central node of Part 3 of the lecture.

| Step | Work                                                          | Time   |
|------|---------------------------------------------------------------|--------|
| 1    | The addresses of the laptop, the Raspberry Pi, and the router | 5 min  |
| 2    | Find the devices, check the Raspberry Pi                      | 5 min  |
| 3    | Throughput, jitter, and loss                                  | 10 min |

```
ip -br addr && ip route                # laptop and Raspberry Pi
python3 ../hardware/HW-09/net_check.py --scan 192.168.8.0/24
python3 ../hardware/HW-09/net_check.py pi-NN.local --need 22
iperf3 -s                               # on the Raspberry Pi
iperf3 -c pi-NN.local -t 10              # laptop to Raspberry Pi
iperf3 -c pi-NN.local -t 10 -R           # Raspberry Pi to laptop
iperf3 -c pi-NN.local -t 10 -u -b 20M    # UDP: jitter and loss
ping -c 100 -i 0.2 pi-NN.local
```

- ▶ **You see:** one line for each device of the scan, and **RESULT: PASS** for the check.
- ▶ **You record:** the receiver bit rates in the two directions, the UDP jitter and lost datagrams, the last two lines of ping.

| Step | Work                               | Time   |
|------|------------------------------------|--------|
| 1    | Start MediaMTX on the Raspberry Pi | 10 min |
| 2    | Check the port and the CPU load    | 5 min  |
| 3    | Open the stream with ffplay or VLC | 10 min |
| 4    | Task C1 in stream_detect.py        | 5 min  |
| 5    | Read the stream in Python          | 10 min |

```
cd ~/mediamtx                                # on the Raspberry Pi
cp mediamtx_cam.yml mediamtx_cam.yml.orig
./mediamtx mediamtx_cam.yml
ss -tln | grep 8554                           # in a second SSH terminal
top -b -n 1 | grep mediamtx
```

- ▶ **You see:** [RTSP] started with listeners on :8554 and [path cam] stream is available and online.
- ▶ ss must show \*:8554 or 0.0.0.0:8554, not 127.0.0.1:8554: the fault of the exercise of Part 1.
- ▶ **You record:** the line of ss and the CPU load of mediamtx. The Raspberry Pi 5 encodes H.264 in software.

```
ffplay -fflags nobuffer -flags low_delay -rtsp_transport tcp \  
    rtsp://pi-NN.local:8554/cam  
PY=~/.day08_env/bin/python # for the rest of the lab  
URL=rtsp://pi-NN.local:8554/cam  
$PY stream_detect.py $URL --seconds 20
```

- ▶ **Task C1:** the method `_store` of `NewestFrameReader` keeps only the newest frame. No list, no queue: a slow model always gets a new frame.
- ▶ The script opens the stream with one decoder thread. Each decoder thread holds one frame (Part 3 of the lecture).

**You see:** Task C1: complete, one line each 2 seconds, and one RESULT line at the end:

```
1280x720 stream ... frames/s processed ... frames/s skipped ...  
RESULT label=run url=rtsp://pi-NN.local:8554/cam reader=newest ...
```

**You record:** the seconds until the first image in `ffplay`, and `stream_fps` and `processed_fps`. With no model, the two rates are almost the same.

# Part C: the detector on the stream (45 min)

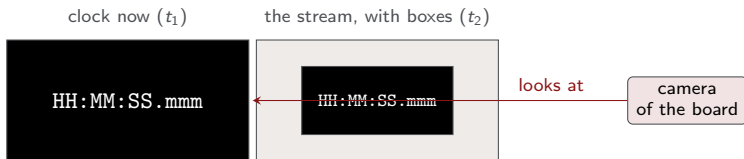
| Step | Work                                                      | Time   |
|------|-----------------------------------------------------------|--------|
| 1    | The detector: yolo11n.pt and your Day 8 model             | 10 min |
| 2    | Latency of setting 1, with no model and with the detector | 15 min |
| 3    | Setting 2: $640 \times 480$ at 0.5 Mbit/s                 | 15 min |
| 4    | The reader in order                                       | 5 min  |

```
$PY stream_detect.py $URL --seconds 30 --show \  
  --model ../day08/models/yolo11n.pt --imgsz 320  
$PY stream_detect.py $URL --seconds 30 --show \  
  --model ../day08/models/cupbottle_320_ncnn_model
```

```
1280x720 stream ... frames/s processed ... frames/s skipped ... model ... ms objects ...
```

- ▶ **You see:** a window with the boxes of the detector.
- ▶ processed is the rate of the model. skipped is the frames that the model did not see: the reader dropped them on purpose.
- ▶ **You record:** the processed rate, the skipped frames each second, and the model time, for the two models.

# Part C: measure the latency with a clock



```
$PY stream_detect.py $URL --clock --label hd_nomodel  
$PY stream_detect.py $URL --clock --label hd_yolo \  
--model ../day08/models/yolo11n.pt --imgsz 320
```

- ▶ Point the camera at the clock on the left. Press s 10 times, about one second apart, then q.
- ▶ In each image latency\_<label>\_NN.png: latency =  $t_1 - t_2$ . It includes the camera, the encoder, the Wi-Fi, the decoder, the model, and the screen.
- ▶ One reading has an error of about 50 ms (a screen of 60 Hz, a camera of 30 frames/s). **You record:** the 10 values, the median, the minimum, and the maximum.

Step 3: stop MediaMTX (Ctrl-C), change three lines of `~/mediamtx/mediamtx_cam.yml`, start it again:

```
rpiCameraWidth: 640  
rpiCameraHeight: 480  
rpiCameraBitrate: 500000
```

Repeat step 2 with the detector and `--label sd_yolo`.

Step 4: read each frame in order, with a slow model:

```
$PY stream_detect.py $URL --seconds 30 --in-order \  
  --model ../day08/models/yolo11n.pt --imgsz 640  
cp mediamtx_cam.yml.orig mediamtx_cam.yml      # then start MediaMTX again
```

- ▶ **You see:** each line ends with behind ... s, the time that the reader is behind the live stream.
- ▶ **You record:** the value after 30 s. Compare it with  $1 - F/30$  seconds for each second, where  $F$  is the processed rate (Part 3 of the lecture).

| Step | Work                                            | Time  |
|------|-------------------------------------------------|-------|
| 1    | Upload sketches/xiao_mjpeg_stream               | 8 min |
| 2    | See the stream in a browser                     | 2 min |
| 3    | Task D1 in mjpeg_rate.py, then the bit rate     | 7 min |
| 4    | The detector and the latency on the XIAO stream | 8 min |

- ▶ Write the Wi-Fi password of the lab in WIFI\_PASSWORD. Select XIAO\_ESP32S3 and Tools > PSRAM > OPI PSRAM.
- ▶ The sketch sends the camera at VGA with JPEG\_QUALITY 12. The camera module makes each JPEG image.
- ▶ The sketch serves one reader at a time. Close the browser tab before you run a script.

**You see** in the Serial Monitor (115200 baud):

```
Camera Stream Ready! Go to: http://192.168.8.163/  
sent ... frames in 5 s: ... frames/s, mean JPEG ... bytes, RSSI ... dBm
```

Source: The address is an example. The sketch is a changed copy of Streaming\_Video.ino of XIAO-ESP32S3-Sense.

```
python3 mjpeg_rate.py http://192.168.8.163/ --seconds 10 --save xiao.jpg  
$PY stream_detect.py http://192.168.8.163/ --clock --label xiao_yolo \  
--model ../day08/models/yolo11n.pt --imgsz 320
```

Task D1: complete

RESULT url=http://192.168.8.163/ seconds=... frames=... fps=... mean\_kb=... mbit\_s=...

- ▶ **Task D1:** the function `stream_stats` returns the frames each second, the mean JPEG size in KB, and the bit rate: all bytes  $\times 8$  / seconds.
- ▶ Check the result: frames/s  $\times$  mean KB  $\times 1024 \times 8$  must give `mbit_s` (the exercise of Part 2).
- ▶ **You record:** the three values, the status line of the sketch, the processed rate of the detector, and 10 latency values.
- ▶ Compare the two streams: the bits for each frame, the frame rate, the latency, and the work of each processor.

The instructor checks this at the end of the lab:

- ▶ ✓ `stream_detect.py` prints Task C1: complete, and `mjpeg_rate.py` prints Task D1: complete.
- ▶ ✓ Live demonstration: the detector draws boxes on the RTSP stream of the Raspberry Pi, on the laptop.
- ▶ ✓ The report gives the latency (10 values and the median) for two settings of the RTSP stream with the detector, and the bit rate and the frame rate of the XIAO stream.
- ▶ ✓ The Decision Log gives numbers and names one trade-off.

**Hand in:** the file `report.md`. Keep the images `latency_*.png`: the instructor can ask for one.

About 100 words: one design decision, your measured numbers, one trade-off.

A shop wants to count the people at its entrance. One camera hangs at the door. A Raspberry Pi 5 stands in the back office, 15 m away, on Wi-Fi. The count must change within 1 second after a person enters, and no image may leave the shop.

- ▶ Where do you run the detector?
- ▶ Which stream settings do you select?
- ▶ What leaves the camera, and what leaves the shop?

A good Decision Log:

- ▶ gives your measured latency and processed rate for the setting that you select, with the method (the clock, 10 values),
- ▶ compares its bit rate with your Wi-Fi throughput,
- ▶ names what you lose: for example image quality against latency.

# If a part does not work, and common problems

| Problem                                                                                          | Fix or fallback                                                                                                                                                                          |
|--------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ERROR: cannot open the stream<br>An error for the camera                                         | Check the log of MediaMTX and <code>ss -tln</code> .<br>Use the address in place of <code>pi-NN.local</code> .<br>A second program uses the camera, for example a Day 8 script. Stop it. |
| The camera does not work                                                                         | Send a test image to the path <code>test</code> (HW-06, step 8)                                                                                                                          |
| ERROR: no file <code>detector.py</code>                                                          | Give the folder: <code>--day08</code><br><code>&lt;path&gt;/day08/pi</code>                                                                                                              |
| No module named <code>'ultralytics'</code>                                                       | Start the script with <code>\$PY</code> , the Day 8 environment                                                                                                                          |
| Grey areas in the image for a short time                                                         | Lost packets with UDP. Use TCP, or write it in the report.                                                                                                                               |
| The latency grows during a run<br><code>mjpeg_rate.py</code> waits, the browser shows the stream | Use the reader of the lab (no <code>--in-order</code> )<br>The sketch serves one reader at a time: close the browser tab                                                                 |
| The XIAO does not connect, or<br><code>Camera init failed</code>                                 | 2.4 GHz and the password. OPI PSRAM, the camera connector.                                                                                                                               |
| A task is not complete in time                                                                   | Use the file of <code>solutions/</code> , and write this in the report                                                                                                                   |

The README of the lab has the complete list.

This lecture adapts material from these sources:

- ▶ *Edge AI Engineering: Raspberry Pi* by Marcelo Rovai.  
[github.com/Mjrovai/EdgeML\\_Made\\_Ease\\_ebook](https://github.com/Mjrovai/EdgeML_Made_Ease_ebook) and  
[github.com/Mjrovai/EdgeML-with-Raspberry-Pi](https://github.com/Mjrovai/EdgeML-with-Raspberry-Pi), GPL-3.0.
- ▶ *TinyML Made Easy: XIAO ESP32S3* by Marcelo Rovai.  
[github.com/Mjrovai/XIAO-ESP32S3-Sense](https://github.com/Mjrovai/XIAO-ESP32S3-Sense), Apache-2.0.
- ▶ *XIAO: Big Power, Small Board* by Lei Feng and Marcelo Rovai.  
[github.com/Mjrovai/XIAO\\_Big\\_Power\\_Small\\_Board-ebook](https://github.com/Mjrovai/XIAO_Big_Power_Small_Board-ebook), GPL-3.0.

Material from a source with the licence CC BY-NC-SA 4.0 keeps that licence.